

Core-Affine Threaded Implementation of a Min-Max Transformation Algorithm on a Matrix

Euan Jed S. Tabamo, *Student, Institute of Computer Science*

I. INTRODUCTION

CORE AFFINITY enables the binding and unbinding of a process or a thread to a central processing unit (CPU) or range of CPUs so that the process of thread will execute only on the designated CPU or CPUs rather than any CPU CITE [1]. Processes or threads are usually pinned to specific cores for several reasons. One reason is reducing the number of cache misses; if a process runs on the same cache it previously ran on, there may be some data in the CPU cache that can be reused. Another reason to pin processes or threads is to limit the amount of cores a CPU-intensive process uses, leaving other cores available for the other programs to use.

II. OBJECTIVES

The primary objective of this lab report is to measure the average running time of core-affine implementations of the min-max transformation algorithm on an $n \times n$ matrix based on the number of threads t running in parallel.

Specifically, the lab report aims to solve the following questions:

- What is the difference of the average time for running the core-affine implementation with one thread compared to the average running time for a serial and non-core-affine parallel implementation?
- Does the average runtime of the core-affine implementation offer an improvement over the non-core-affine parallel implementation for all thread counts? At how many threads is the average runtime of the core-affine implementation better? Is the average runtime statistically the same or slower?
- What occurs when the matrix is subdivided into submatrices, but in a manner where each submatrix is of the size $\frac{n}{t} \times n$?

III. METHODOLOGY

The programming language used is the Rust programming language, version 1.94.0, on a personal machine with an Intel Core i5-12500H and 16GB of DDR4 RAM on Windows 11 25H2. The program was created, compiled, and run on Windows 11 25H2.

The program requires an integer n as an argument, creating and filling a 2-dimensional $n \times n$ matrix X with random 32-bit integers from the range $[1, 100]$ cast to 64-bit floating point numbers. It also requires the specific implementation to

run such as serial, parallel-by-column, parallel-by-row, core-affine-column, and core-affine-row.

The program then applies the min-max transformation on X depending on the implementation. Regardless of the implementation, the algorithm applies every element in each column of the matrix using the following formula:

$$X_{ij} = \frac{X_{ij} - \min(X_j)}{\max(X_j) - \min(X_j)}$$

The serial implementation iterates over every column X_j in X , finds, and stores the minimum and maximum values in X_j . It then iterates over X_j and transforms every element X_{ij} in-place. The parallel-by-column implementation instead divides the $n \times n$ matrix into chunks of $n \times \lceil \frac{n}{t} \rceil$, which are then distributed to t threads. The parallel-by-row implementation divides the $n \times n$ matrix into chunks of $\lceil \frac{n}{t} \rceil \times n$ chunks, which are then distributed to t threads. If the size of the square matrix does not cleanly divide into the number of threads, the last thread processes the remaining chunk.

The program then prints the running time of the chosen implementation of the min-max transformation algorithm to standard output.

The program uses the rand crate to generate random numbers needed to fill the matrix.

Lastly, the program was run in release mode, recording the average runtime using input size $n = 25000$ and various number of threads t .

IV. RESULTS AND DISCUSSION

TABLE I
SERIAL MIN-MAX TRANSFORMATION RUNTIME

Runtime (in seconds) at $n = 25,000$			
Run 1	Run 2	Run 3	Average
1.741521	1.774573	1.735985	1.750693

TABLE II
PARALLEL-BY-COLUMN MIN-MAX TRANSFORMATION RUNTIME

Runtime (in seconds) at $n = 25,000$				
t	Run 1	Run 2	Run 3	Average
1	1.731572	1.720737	1.778360	1.743556
2	0.930175	0.930205	0.985208	0.948529
4	0.587133	0.552351	0.552065	0.563850
8	0.495859	0.451775	0.450315	0.465983
16	0.484385	0.455715	0.447944	0.462682
32	0.495004	0.455520	0.463635	0.471386
64	0.489310	0.454250	0.452574	0.465378

Euan Jed S. Tabamo is with the University of the Philippines Los Baños, under the Institute of Computer Science.

Laboratory Report submitted on March 26, 2026.

TABLE III
PARALLEL-BY-ROW MIN-MAX TRANSFORMATION RUNTIME

Runtime (in seconds) at $n = 25,000$				
t	Run 1	Run 2	Run 3	Average
1	1.896368	1.867805	1.839865	1.868013
2	1.153397	1.134970	1.156738	1.148368
4	0.926942	0.864041	0.839406	0.876796
8	0.827960	0.756654	0.792514	0.792376
16	0.743120	0.719265	0.731253	0.731213
32	0.808732	0.756357	0.735950	0.767013
64	0.847657	0.798897	0.813148	0.819901

TABLE IV
CORE-AFFINE PARALLEL-BY-COLUMN MIN-MAX TRANSFORMATION RUNTIME

Runtime (in seconds) at $n = 25,000$				
t	Run 1	Run 2	Run 3	Average
1	1.748151	1.743473	1.770424	1.754016
2	1.052664	1.014215	1.021102	1.029327
4	0.586647	0.560095	0.597293	0.581345
8	0.487071	0.432598	0.414312	0.444660
16	0.549224	0.540138	0.513324	0.534229
32	0.530370	0.475022	0.545878	0.517090
64	0.512913	0.507278	0.485229	0.501806

TABLE V
CORE-AFFINE PARALLEL-BY-ROW MIN-MAX TRANSFORMATION RUNTIME

Runtime (in seconds) at $n = 25,000$				
t	Run 1	Run 2	Run 3	Average
1	3.048935	2.863647	2.542762	2.818448
2	1.619974	1.631595	1.563739	1.605102
4	1.075562	1.048194	1.036214	1.053323
8	0.853904	0.940799	0.840839	0.878514
16	0.929454	0.953848	0.956115	0.946472
32	0.912855	0.900632	0.990402	0.934630
64	0.998225	0.956770	1.514853	1.156616

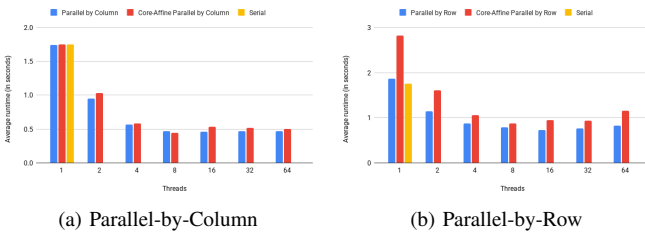


Fig. 1. Chart of the average runtime of the program as t grows ($t = 1 \dots 64$).

Table I, Table II, Table III, Table IV, Table V, and Figure 1 show the average runtime of serial, parallel, and core-affine implementations of the min-max transformation algorithm on a 25000×25000 matrix. Based on these data, it can be noted that:

- The runtime of the serial implementation is marginally faster than a ‘parallel’ implementation with one thread.
- The runtime of the parallel implementation decreases as t grows up to a $t = 16$.

- The runtime of the parallel-by-column implementation stagnates after $t = 16$ while the runtime of the parallel-by-row implementation marginally increases.

TABLE VI
PARALLEL-BY-COLUMN MIN-MAX TRANSFORMATION RUNTIME
($t = 16, t = n/8, t = n/4, t = n/2, t = n$)

Runtime (in seconds) at $n = 25,000$				
t	Run 1	Run 2	Run 3	Average
16	0.484385	0.455715	0.447944	0.462682
3125	0.619442	0.502341	0.516202	0.545995
6250	0.689470	0.624403	0.626738	0.646870
12500	1.088619	0.923421	0.942037	0.984692
25000	1.708960	1.595630	1.750090	1.684893

TABLE VII
PARALLEL-BY-ROW MIN-MAX TRANSFORMATION RUNTIME
($t = 16, t = n/8, t = n/4, t = n/2, t = n$)

Runtime (in seconds) at $n = 25,000$				
t	Run 1	Run 2	Run 3	Average
16	0.743120	0.719265	0.731253	0.731213
3125	3.652691	3.390796	3.528734	3.524074
6250	7.111224	6.822365	6.399920	6.777837
12500	19.269385	15.474814	14.965153	16.569784
25000	119.347282	126.585121	146.911751	130.948051

TABLE VIII
CORE-AFFINE PARALLEL-BY-COLUMN MIN-MAX TRANSFORMATION RUNTIME
($t = 16, t = n/8, t = n/4, t = n/2, t = n$)

Runtime (in seconds) at $n = 25,000$				
t	Run 1	Run 2	Run 3	Average
16	0.549224	0.540138	0.513324	0.534229
3125	0.623607	0.561960	0.545529	0.577032
6250	0.703126	0.653321	0.653500	0.669982
12500	1.032701	0.928341	0.960538	0.973860
25000	1.763315	1.964059	1.643959	1.790444

TABLE IX
CORE-AFFINE PARALLEL-BY-ROW MIN-MAX TRANSFORMATION RUNTIME
($t = 16, t = n/8, t = n/4, t = n/2, t = n$)

Runtime (in seconds) at $n = 25,000$				
t	Run 1	Run 2	Run 3	Average
16	0.929454	0.953848	0.956115	0.946472
3125	5.527178	5.159348	4.738725	5.141750
6250	10.434873	8.463778	8.544150	9.147600
12500	21.268092	20.489557	22.553900	21.437183
25000	120.689541	131.996348	167.333570	140.006486

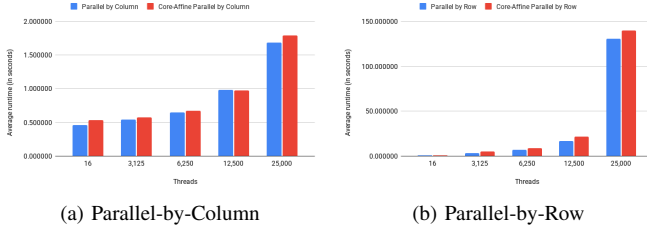


Fig. 2. Chart of the average runtime of the program at extreme thread counts ($t = 16 \dots 25000$).

Table VI, Table VII, Table VIII, Table IX, and Figure 2 show the average runtime of parallel and core-affine implementations of the min-max transformation algorithm at higher t values approaching n . Based on these data, it can be additionally noted that:

- The parallel implementations suffer decreased performance as t approaches n .
- The parallel-by-row implementation suffers much more heavily as t approaches n compared to the parallel-by-column implementation.

A. What is the difference of the average time for running the core-affine implementation with one thread compared to the average running time for a serial and non-core-affine parallel implementation?

At $t = 1$, the core-affine implementation is consistently slower than the non-core-affine parallel implementation. While the column-wise implementation's results are marginal, the row-wise implementation's results show drastic differences, with core-affine having 2.818 seconds on average compared to serial's 1.75 seconds on average and non-core-affine parallel version's 1.744 seconds on average.

B. Does the average runtime of the core-affine implementation offer an improvement over the non-core-affine parallel implementation for all thread counts? At how many threads is the average runtime of the core-affine implementation better? Is the average runtime statistically the same or slower?

Core-affinity does not offer an improvement for all thread counts. It is slower in the majority of tested scenarios.

The core-affine implementation performs slightly better only at $t = 8$ (0.444s vs 0.465s) and in the more extreme case of $t = 12,500$ (0.973s vs 0.984s). In these cases, the core-affine implementation does not win by a significant amount. The core-affine implementation is either slower or statistically similar for column-wise partitioning. The core-affinity implementation is significantly slower across all thread counts for row-wise partitioning. This is most likely due to a combination of overhead from pinning the thread and the inherent inefficiency of row-wise access in a column-major matrix.

C. What occurs when the matrix is subdivided into submatrices, but in a manner where each submatrix is of the size $(\frac{n}{t} \times n)$?

The parallel-by-row and core-affine-row implementations have significantly worse performance compared to column-

wise partitioning $(n \times \frac{n}{t})$. The program's underlying matrix object is stored as a 1-dimensional column-major vector. As such, partitioning by column assigns contiguous blocks of physical memory to each thread maximizing spatial locality [2]. This leads to the parallel-by-column implementation yielding fast runtimes. In contrast, the parallel-by-row implementation forces threads to read across the columns in a strided access pattern, leading to cache misses and slower runtimes.

The core-affine row-wise implementation is the slowest of all tested methods due to the combination of strided memory access and affinity overhead. We can note that pinning threads in the core-affine row-wise implementation provides no benefit from the cache because the data is physically scattered across memory anyway.

V. CONCLUSION

The results of this research activity demonstrate that core-affinity, while having valid purposes, does not guarantee a performance gain in all scenarios. The core-affine implementations perform statistically similar at best to the non-core-affine implementations, and are significantly worse than non-core-affine implementations at worst.

APPENDIX A SOURCE CODE

The source code is available at the following link:
<https://github.com/ultbmo/cmsc180/tree/main/exer03>

REFERENCES

- [1] "Processor affinity," Dec. 2025, page Version ID: 1327825361. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Processor_affinity&oldid=1327825361
- [2] "Locality of reference," Sep. 2025, page Version ID: 1311654893. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Locality_of_reference&oldid=1311654893

Euan Jed S. Tabamo is an undergraduate BS Computer Science student in the University of the Philippines Los Baños. He is an aspiring software developer and game developer.

